[Khanh Pham] Userspace Debugging Toolkit: GDB, strace & Valgrind

Debug tools

> In this document, I will write about 3 debug tools: `gdb`, `strace` and `valgrind`.

GDB

GDB is the most important to debug with embedded developers.

GDB help us interact inside the program when it running. That mean we can stop the program wherever we want, display the value of variables, register and show the change of them.

Before going to an example of using gdb tools, I will list the gdb cheatsheet:

1. Running the Program

```
(gdb) run / r                    run the program
(gdb) run args... / r args...    run with command-line arguments
(gdb) set args args...           set arguments for the next run
(gdb) show args                  show current program arguments

(gdb) continue / c               continue execution
(gdb) step / s                   step into the next source line/function
```

```
(gdb) step count                step count times
(gdb) next / n                  step over the next source line/function
(gdb) next count                step over count times

(gdb) finish / fin              run until the current function returns
(gdb) CTRL-C                    interrupt the running program
(gdb) kill / k                  terminate the running program

(gdb) attach PID                attach GDB to a running process
(gdb) detach                    detach from the process
```

2. Stack Backtrace

```
(gdb) backtrace / bt            show the call stack
(gdb) bt                        short form of backtrace

(gdb) frame / f                 show the current stack frame
(gdb) frame N                   switch to stack frame N
(gdb) up                        move to the caller's frame
(gdb) down                      move to the called function's frame

(gdb) info locals / i locals    show local variables
(gdb) info args / i args        show function arguments
```

3. Browsing Source Code

```
(gdb) list / l                  show source code around current line
(gdb) list 101                  show source around line 101
(gdb) list 1,10                 show lines 1–10
(gdb) list main                 show source around main()
(gdb) list file.c:main          show main() from file.c
(gdb) list -                    show previous source lines
(gdb) list *0xADDRESS           show source corresponding to address

(gdb) cd DIR                    change working directory
(gdb) pwd                       show current working directory
```

```
(gdb) search REGEXP              search forward for a regular expression
(gdb) reverse-search REGEXP      search backward for a regular expression

(gdb) dir DIR                    add directory to source search path
(gdb) dir                        reset source search path
(gdb) show directories           show source search path
```

4. Inspecting Variables

```
(gdb) print / p EXPR             print the value of an expression
(gdb) print/x EXPR               print value in hexadecimal
(gdb) print/t EXPR               print value in binary
(gdb) print/d EXPR               print value in decimal

(gdb) info locals / i locals     show local variables
(gdb) info args / i args         show function arguments

(gdb) ptype EXPR                 show the type definition
(gdb) whatis EXPR               show the type of an expression

(gdb) set variable x = VALUE     change the value of variable x

(gdb) display EXPR               automatically print EXPR when execution stops
(gdb) undisplay                  remove a display
(gdb) info display               show active displays
```

5. Inspecting Memory

```
(gdb) x/FORMAT ADDRESS            examine memory at ADDRESS
(gdb) x/10xw ADDRESS              show 10 words in hexadecimal
(gdb) x/16xb ADDRESS              show 16 bytes in hexadecimal
(gdb) x/10dw ADDRESS              show 10 words in decimal
(gdb) x/s ADDRESS                 show memory as a string
(gdb) x/10i ADDRESS               show 10 assembly instructions

(gdb) help x                      show help for memory examination
```

6. Arrays & Value History

```
(gdb) print / p array[i]@count    print count elements starting at array[i]

(gdb) print $                     print the previous value
(gdb) print $1                    print value #1 from value history
(gdb) print $2                    print value #2 from value history

(gdb) show values                 show saved values
```

7. Finding Functions & Variables

```
(gdb) info functions / i functions        list functions
(gdb) info functions REGEXP               list functions matching REGEXP

(gdb) info variables / i variables        list global/static variables
(gdb) info variables REGEXP               list variables matching REGEXP

(gdb) ptype TYPE                          show type definition
(gdb) whatis EXPR                         show type of expression
```

8. Watchpoints

```
(gdb) watch EXPR                  stop when EXPR changes
(gdb) rwatch EXPR                 stop when EXPR is read
(gdb) awatch EXPR                 stop when EXPR is read or written

(gdb) info watchpoints            show active watchpoints
(gdb) delete N                    delete watchpoint N
```

9. Object File / Executable

```
(gdb) file PROGRAM                load executable for debugging
(gdb) file                         unload current executable

(gdb) symbol-file FILE            load symbol/debug information
```

```
(gdb) exec-file FILE              specify executable to run

(gdb) core-file CORE              load a core dump for post-mortem debugging
```

10. Signals

```
(gdb) info signals                show signal handling configuration

(gdb) handle SIGNAL ACTION        configure how GDB handles a signal

(gdb) handle INT stop             stop when SIGINT occurs
(gdb) handle INT nostop           don't stop on SIGINT

(gdb) handle INT print            print a message when SIGINT occurs
(gdb) handle INT noprint          don't print a message

(gdb) handle INT pass             pass SIGINT to the program
(gdb) handle INT nopass            don't pass SIGINT to the program

(gdb) signal SIGNAL               continue and send SIGNAL to the program
(gdb) signal 0                    continue without sending a signal
```

11. Machine-Level Debugging

```
(gdb) info registers / i registers       show CPU registers
(gdb) info all-registers                  show all registers

(gdb) print/x $rax                        print register RAX in hexadecimal
(gdb) print/x $pc                         print program counter

(gdb) stepi / si                          execute one machine instruction
(gdb) si                                  short form of stepi

(gdb) nexti / ni                          execute one instruction, stepping over calls
(gdb) ni                                  short form of nexti

(gdb) display/i $pc                       automatically show current instruction
```

```
(gdb) info line 22                          show machine-code addresses for source line 22
(gdb) info line *0xADDRESS                  show source line corresponding to an address

(gdb) x/10i main                            show 10 instructions starting at main
(gdb) disassemble / disas main              disassemble a function
(gdb) disassemble / disas ADDRESS           disassemble around an address
```

12. C++ Debugging

```
(gdb) break / b Class::method               break at a C++ member function
(gdb) list / l Class::method                show source of a member function
(gdb) ptype Class                           show class definition

(gdb) print / p *this                       print the current object

(gdb) rbreak REGEXP                         set breakpoints using a regular expression
```

13. Command History & Editing

```
(gdb) show commands                         show previously executed GDB commands
(gdb) set history expansion on              enable command-history expansion

(gdb) ↑ / ↓                                 browse command history
(gdb) RETURN                                repeat the previous command
```

14. TUI / User Interface

```
(gdb) tui enable                             enable TUI mode
(gdb) tui disable                            disable TUI mode

(gdb) layout src                             show source-code window
(gdb) layout asm                             show assembly window
(gdb) layout split                           show source + assembly
(gdb) layout regs                            show registers
```

```
(gdb) focus WINDOW                  select the active TUI window
(gdb) refresh                       refresh the TUI display
(gdb) info win                      show TUI windows
```

15. Miscellaneous

```
(gdb) define NAME ... end           define a custom GDB command
(gdb) shell COMMAND                 execute a shell command

(gdb) source FILE                   load and execute GDB commands from a file

(gdb) help COMMAND                  show help for a GDB command
(gdb) quit / q                      exit GDB
```

Core dump

The default action of certain signals is to cause a process to terminate and produce a core dump file, a file containing an image of the process's memory at the time of termination

Normally, linux set the number of core dump file is 0, so we need to change it by this command:

```
ulimit -c unlimited
```

We can also change the place where the core dump file stored is the folder of source main by this command:

```
sudo sysctl -w kernel.core_pattern=core
```

An example of using `gdb` to fix

> Now let to fix unknow error segmentation fault by using coredump file. This program example is writed by GEMINI.

Code source: https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/tree/main/gdb/core_dump_coffee

- After run the program, we can see the result:

```
------------------------------------------------
Subtotal       : $  4.05
Member Discount: -$ 0.00
Tax (8%)       : $  0.32
FINAL TOTAL    : $  4.37
Segmentation fault (core dumped)
```

- Then, the core dump file is generated. Using gdb to open this file.

```
Core was generated by `./cafe_pos'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x0000631f478a5feb in format_loyalty_footer (order=0x631f78dea0f0,
    buffer=0x7ffcab33af40 '-' <repeats 52 times>, "\nLoyalty Card   : BC-1105\nMembership
Tier: BRONZE\nPoints Earned  : +2   | New Balance: 17 pts\n", '-' <repeats 52 times>,
"\nT"..., max_len=512) at order_manager.c:130
130□    int current_points = card->accumulated_points;
(gdb)
```

WE can see the error from the line 130 at file `order_manager.c`.

- We use the command `bt full` to see all backtrace of this line:

```
#0  0x0000631f478a5feb in format_loyalty_footer (order=0x631f78dea0f0,
    buffer=0x7ffcab33af40 '-' <repeats 52 times>, "\nLoyalty Card   : BC-1105\nMembership
Tier: BRONZE\nPoints Earned  : +2   | New Balance: 17 pts\n", '-' <repeats 52 times>,
"\nT"..., max_len=512) at order_manager.c:130
        cust = 0x631f78de9830
        card = 0x0
        current_points = 15
        updated_balance = 1
        tier_str = 0x4010333333333333 <error: Cannot access memory at address
0x4010333333333333>
#1  0x0000631f478a6113 in process_and_print_receipt (order=0x631f78dea0f0) at
order_manager.c:161
        receipt_body = '=' <repeats 16 times>, " DAILY BREW CAFE ", '=' <repeats 16 times>,
"\nReceipt #: ORD-0104\nCustomer : David (Walk-in Guest)\n", '-' <repeats 52 times>, "\n Iced
Americano       (L) x1  $  4.05\n   [Su"...
        loyalty_footer = '-' <repeats 52 times>, "\nLoyalty Card   : BC-1105\nMembership Tier:
BRONZE\nPoints Earned  : +2   | New Balance: 17 pts\n", '-' <repeats 52 times>, "\nT"...
#2  0x0000631f478a5518 in simulate_morning_rush () at main.c:40
        alice = 0x631f78de96b0
```

```
        bob = 0x631f78de9730
        charlie = 0x631f78de97b0
        david = 0x631f78de9830
        ord1 = 0x631f78de9880
        ord2 = 0x631f78de9b50
        ord3 = 0x631f78de9e20
        ord4 = 0x631f78dea0f0
#3  0x0000631f478a5598 in main (argc=1, argv=0x7ffcab33b6d8) at main.c:59
No locals.
```

Now we can see exactly the error come from the value of pointer card. This pointer point to `NULL`, so using this pointer is invalid.

- After that, we look for the way to fix this problem. We can see both caller functions(frame #0 and frame #1) are `format_loyalty_footer` and `process_and_print_receipt` is the general function. That mean we should use them for all customer. Therefore, we shouldn't add more functions because it will make the program become more complex. And we also cannot change the value of card to `non-NULL` because the requiment of this program.

> As a result, we can make a handle logic of the frame #0 function, like this:

```
static void format_loyalty_footer(const Order *order, char *buffer, size_t max_len) {
    const Customer *cust = order->customer;
    const LoyaltyCard *card = cust->loyalty_card;

    if(order->customer->loyalty_card == NULL) {
        snprintf(buffer, max_len,

              "Thank you for enjoying your coffee with us!\n"
              "=======================================================");
    }
    else
    {/* Reward points update calculation */
    int current_points = card->accumulated_points;
    int updated_balance = current_points + order->points_earned;

    const char *tier_str = "BRONZE";
    if (card->tier == TIER_GOLD) {
        tier_str = "GOLD (15% OFF)";
```

```
    } else if (card->tier == TIER_SILVER) {
        tier_str = "SILVER (10% OFF)";
    }
```

Thread debugging

In the knowledge above, we know the way to debug in single thread program. In this section, I will write about the multi-thread debugging.

Before going to the example of thread debugging, here is the cheat sheet in this section:

Thread infomation and navigation

```
(gdb) info threads / i th                list all active threads (* indicates active thread)
(gdb) thread <ID> / t <ID>                switch context to thread <ID> (e.g., thread 2)
(gdb) thread find <regex>                search threads matching name or regex
(gdb) thread name <name>                 assign a custom name to the current thread
```

Inspecting thread state and call backs

```
(gdb) backtrace / bt                     show call stack for current thread
(gdb) thread apply all bt / taa bt       show call stack for ALL threads (great for
deadlocks)
(gdb) thread apply <ID1 ID2> bt          show call stack for specific threads (e.g., 1 3)
(gdb) thread apply all print <var>       print value of a variable across all threads
```

Thread execution and scheduling control

```
(gdb) set scheduler-locking on           lock execution strictly to current thread (others
frozen)
(gdb) set scheduler-locking off          allow all threads to run freely (default)
(gdb) set scheduler-locking step         lock other threads during step/next only
(gdb) show scheduler-locking             display current scheduler-locking mode
```

Thread-specific breakpoint

```
(gdb) break <line/func> thread <ID>      set breakpoint for a specific thread only
(gdb) break <line/func> thread <ID> if <cond>  set thread-specific conditional breakpoint
```

Thread event and non-stop mode

```
(gdb) set print thread-events on/off      toggle thread creation/exit messages
(gdb) set non-stop on                     pause only the thread hitting a breakpoint (async
mode)
```

multi-process debugging

```
(gdb) set follow-fork-mode parent/child      Follow the parent/child process after forking
(gdb) set detach-on-fork on                  Detach unfollowing process
(gdb) set detach-on-fork off                 Keep unfollowing process
(gdb) info inferiors                         List all process in this program
(gdb) inferior (number)                      Switch to number process
```

Revision #3
Created 2026-09-21 02:44:48 UTC by Khanh Pham
Updated 2026-09-22 03:54:04 UTC by Khanh Pham